

# Track<sup>2</sup>Net: A Fast Lightweight Model With Keypoint Alignment and Track Anchors Identification for Railway Line Tracking

Haipeng Guo, Xiukun Wei<sup>ID</sup>, *Member, IEEE*, Yang Ji, Chengyu Ge, Qingfeng Tang<sup>ID</sup>, and Kunlin Cai<sup>ID</sup>

**Abstract**—High-speed railways are widely used worldwide due to their high speeds, punctuality, and comfort. Autonomous operation and automatic obstacle avoidance technologies are the key to enhancing operational efficiency and ensuring passenger safety. By identifying the tracks, high-speed rail clearance can be effectively detected, which helps to determine the specific space through which the train travels. In this paper, a lightweight track line tracking model (Track<sup>2</sup> Net) is proposed, which is based on keypoint alignment and track anchor identification. Firstly, the method mathematically analyses the shapes of tracks in non-turnout scenes and then implements a rapid track matching algorithm based on keypoint alignment. For complex track shapes that fail to match, the Complex Track Positioning Module (CTPM) is proposed. The module extracts features by pre-defining the track anchor lines and has better robustness in complex scenes. Additionally, a post-treatment track method is proposed for turnouts to further refine the detection of the track segment occupied by the train. In order to judge the accuracy of track detection more rigorously, improved evaluation metrics are proposed. On the track datasets of the train operation constructed in this paper, Track<sup>2</sup> Net achieves an accuracy of 91.43% and processes images at a rate of 236 FPS. Experiments verify the effectiveness of the model in track line detection.

**Index Terms**—Railway line tracking, keypoints, deep learning, autonomous operation, railway scene understanding, railway safety.

## I. INTRODUCTION

HIGH-SPEED railways are emerging as indispensable parts of modern transportation systems, with the imperative to guarantee the secure operation growing increasingly each day. The application of autonomous operation and automatic obstacle avoidance technologies in high-speed railways is gaining popularity and represents a significant trend in the global development of high-speed rail. At present, many high-speed railways operate at a speed of 350 km/h. This poses a significant challenge to the speed of detection algorithms associated with automatic obstacle avoidance technology. Consequently, intelligent perception for high-speed trains is of paramount

importance. During high-speed rail operations, images captured by onboard cameras contain abundant environmental information. However, the high-speed railway clearance area only constitutes a portion of the image. Background elements within the images are significant sources of interference. Track detection can effectively delineate the high-speed railway clearance area. This technology not only offers clear guidance for train operations but also significantly simplifies the obstacle detection process. By precisely defining the track boundaries, the system can focus on image changes within this specific region more intently, thereby enabling rapid identification and warning of potential obstacles. Therefore, it is essential to examine track geometry under high-speed railway conditions and to devise a rapid and efficient algorithm for track positioning to prevent foreign objects from entering the high-speed railway boundaries and facilitate autonomous operation.

In the early days, to solve the problem of track line detection, the main method is to install monitoring and testing equipment in accident-prone areas. However, track line detection and foreign object intrusion are highly unpredictable, making timely detection of foreign objects infeasible through manual inspection and passive prevention alone. Consequently, vision-based intelligent detection technology emerges. As shown in the following, the main methods are based on image processing and deep learning.

### A. Methods Based on Image Processing

With the advancement of computer vision, the detection of railway track lines is increasingly being processed using traditional image processing techniques. An algorithm leveraging dynamic programming [1] is employed to ascertain the optimal pixel path within the gradient image, subsequently extracting the track ahead of the train. Furthermore, a method predicated on the linear edge distribution within images is proposed [2]. The rail hypothesis is formulated by considering the distribution of linear edges in the image and the predefined rail gauge. By comparing the distance and direction discrepancies between various pairs of rail hypothesis and identifying the optimal match within a specified tolerance range, the actual rail track can be determined. Furthermore, the edge detection method mentioned in [3] is applied for the estimation of the position of the track, determining the clear space in the image along the edge of the track to align with the curve of the edge of the track. A novel method for railway track detection and turnout identification, predicated on HOG (Histogram of

Received 22 December 2024; revised 3 April 2025 and 25 July 2025; accepted 31 October 2025. Date of publication 10 November 2025; date of current version 26 December 2025. This work was supported by the National Key Research and Development Program of China: Research on High-Speed Train Safety Assurance and Maintenance Technologies Based on Active Safety under Grant 2022YFB4301204. The Associate Editor for this article was X. Jia. (*Corresponding author: Xiukun Wei.*)

The authors are with the School of Traffic and Transportation and the Frontiers Science Center for Smart High-Speed Railway System, Beijing Jiaotong University, Beijing 100044, China (e-mail: xkwei@bjtu.edu.cn).

Digital Object Identifier 10.1109/TITS.2025.3628655

1524-9050 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Access provided by VTU consortium. Downloaded on February 14, 2026 at 13:05:04 UTC from IEEE Xplore. Restrictions apply.

Oriented Gradients) features, is proposed [4]. This method first calculates the HOG feature and establishes the integral image, and then extracts the railway track through the region growing algorithm. Then, on the basis of identifying the opening direction of the turnout, the path that the train passes is found. A new track reconstruction algorithm based on graph theory [5] combines inverse perspective mapping and prior geometric knowledge of track to find possible track segments. Based on the Gabor wavelet filtering method [6], the video frame is divided into four regions and filtered by the Gabor wavelet, respectively. Taking into account the size and direction range of the track at each partition, the response of the Gabor wavelet is tuned to a specific frequency to realise the enhancement of the track edge and noise filtering. A convenient and fast method [7] based on the line segment detector (LSD) and the least squares curve fitting is applied to identify the rail in the image. The method of visual railway detection based on superpixels [8] uses SVM classifier to make a decision on superpixels by using the prediction of the features in superpixels, and divides all superpixels predicted as positive railways to be detected. Automatic track detection algorithm based on Hu Moment Invariants [9] uses a B-spline curve as the fitting model. In another study, the track is understood as a set of parabolic segments [10] from the normal perspective, and each new segment is selected from the range of parabolic variations based on its previous segment, thus completing the detection of the track line.

Track detection technology that heavily utilises traditional image processing is heavily based on manually designed features. This approach, which involves designing and detecting track features manually, is only effective in an ideal track environment. In complex real-world scenarios, extracting track features becomes challenging, and detection is unfeasible under uneven illumination or in a blurry track environment, making real-time application difficult.

## B. Methods Based on Deep Learning

1) *Methods Based on Image Segmentation:* In light of the challenges, many scholars begin research into track detection technology that takes advantage of deep learning, achieving significant advancements. To delineate the operational space, the majority of studies employ image segmentation techniques to identify the track area. Specifically, image segmentation facilitates the differentiation of track pixels from background pixels, enabling the precise localization of track lines [40], [41], [43].

On the basis of classical CNN pixel-level classification of rail areas, an improved polygon fitting method is proposed to optimize the extracted contour of rail areas [11] in order to obtain a better contour of rail areas. RailNet [12] provides an end-to-end solution that combines feature extraction and segmentation. The backbone network is modified to extract multiple convolution features, and the pyramid structure is used to spread the features from top to bottom. A railway track segmentation framework [13] with high resolution image is proposed and accelerated by using two neural networks. TPE-Net [14] extracts and associates the complete convolutional encoder-decoder architecture of the left and right track pixels

of each track path instance on the traditional segmentation network.

As a slender object, the rail is similar to the lane line in road traffic. Many track detection methods are derived from automatic detection of the lane line in traffic. Therefore, it is of great significance to review the development of lane detection for the design of track positioning algorithms. SCNN [15] regards the rows or columns of the feature map as layers, and performs convolution, nonlinear activation and other operations in turn to form a deep neural network. In this way, information can spread between neurones in the same layer. Furthermore, in the case of discontinuous or chaotic objects, SCNN can keep the lane line smooth and continuous. An information transfer method [16] is proposed to capture the spatial relationship of pixels between rows and columns, and according to this information transfer method, a feature enhancement module RESA is designed to improve the computational efficiency and aggregate spatial information effectively. CurveLane NAS [17] uses lane-sensitive architecture search and adaptive point mixing to automatically fuse and capture long-term coherent and curve information, and make more effective calculation and allocation.

Using segmentation-based railway line detection methods can provide higher accuracy. But they need more computing resources, many model parameters and low processing speed. To a large extent, this is due to the prediction of the whole image at the pixel level. At the same time, due to lack of prior knowledge of the overall shape of the track line, the detection results based on segmentation are easily affected by unfavourable conditions, such as occlusion and bad lighting conditions.

2) *Methods Based on Parameter Estimation:* The image is a 2D plane, and the track line is a straight line or a curve, which can be described by a polynomial. Polynomial parameters can be learnt directly using the deep learning model, and track detection location can be completed.

PolyLaneNet [18] obtains an input image from a forward-looking camera installed in a vehicle, and outputs a polynomial representing each lane in the image and a confidence score. LSTR [19] can directly output the lane shape model parameters and use the network built by the transformer to learn more abundant structure and context. Parameter-based methods have faster inference speed than other methods. Because the convolution operation of the methods is lower and the network complexity is lower, parameter-based methods have more advantages in practical applications, especially in scenarios with limited computing resources. However, parameter-based methods exhibit inadequate performance when dealing with complex track lines.

3) *Methods Based on Keypoints Identification:* At present, the application of keypoint-based methods in lane and track line detection is attracting more and more attention. By conceptualizing the track into an array of many sampling points, this method can describe and understand the morphological features of the track line well, thus improving the adaptability to complex conditions. During the detection process, it is necessary to regress the offset between each sampling point and the keypoint to locate the position of the track lines.

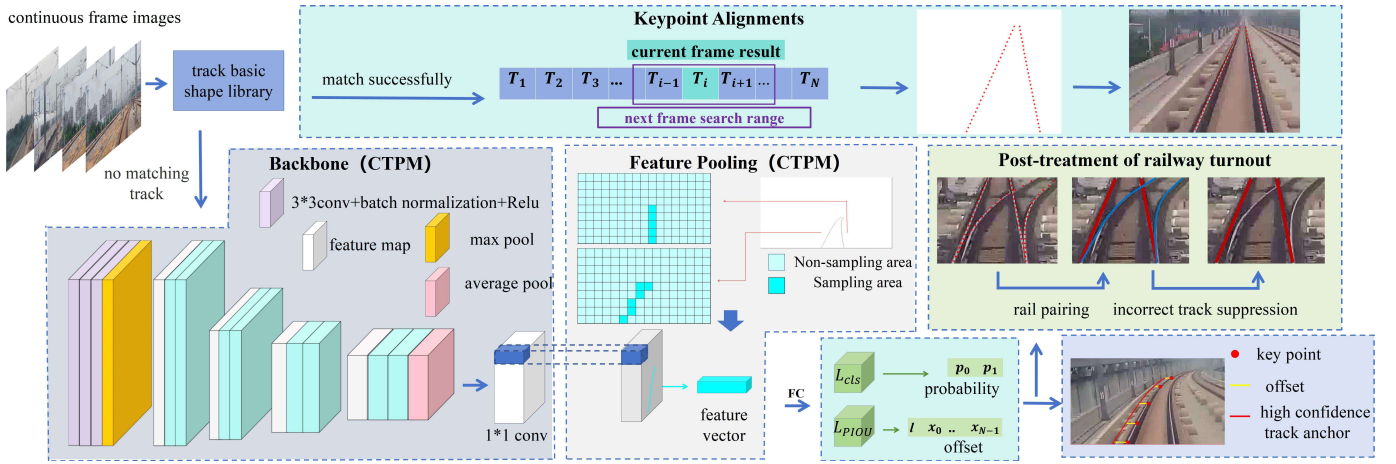


Fig. 1. Track<sup>2</sup>Net Detection Framework, including the keypoint alignment, the CTPM and the Post-treatment of railway turnouts.

A topology-based track detection method [20] is used to obtain the pixels of the rail. The track pixels are transformed into the keypoints of the track and the topological relationships between the keypoints. The railway lane is reconnected through this topological relationship. Finally, the geometric features of the rail are used to match the rail. Line-CNN employs a line proposal mechanism [21], which involves rays emanating from the left, lower, and right boundaries at various angles, to segment the lane lines into positive and negative samples. This approach facilitates the calculation of the loss and the subsequent updating of the model parameters. Furthermore, LaneATT introduces an innovative anchor-based attention mechanism [22] that aggregates global information, thereby enhancing both the accuracy and the processing speed of the system. Meanwhile, lane detection methods based on row anchors are also widely used [23], and lightweight backbones are used to achieve high inference speed. In order to extract features of different scales, a top-down lane detection framework CondLaneNet [24] is proposed. The method first detects lane instances, and then predicts the line shape of each instance dynamically. To solve the problem of lane instance-level recognition, a conditional lane detection strategy based on conditional convolution and a line-by-line formula is proposed. CLNet [25] first performs detection in high semantic features to locate lanes roughly. Then, the refinement is based on fine detail features to obtain a more precise position. High-precision detection results can be obtained by gradually refining the position and feature extraction of lanes. A new keypoints-based lane detection model GANet [26] uses a new modeling method. The starting point is used to represent each lane, and the offset from each keypoint to the starting point of the lane line is predicted. The keypoints of the predicted starting point in the same neighborhood belong to the same lane instance. The shape of the lane line can be restored by sequentially connecting these points.

The method based on keypoints can better capture the detailed features of track lines, while using non-maximum suppression is helpful to screen out results with the highest confidence. The combination of these methods makes track line detection achieve better results in complex environment.

In this paper, the track where the train is running in the high-speed railways scene is taken as the research object, and the image is captured by the front camera of the high-speed railway train. Based on image processing technology and deep learning methods, the railway line detection model is designed, and finally, the purpose of real-time detection of the current track area of the train for the high-speed railway is achieved. The overall detection framework is shown in Fig. 1. The main contributions of this study can be summarized as follows.

1) Based on keypoint alignment and track anchor identification, an efficient method named Track2Net is proposed to quickly track lines. By making full use of the prior knowledge of the track shapes, Track<sup>2</sup>Net has higher precision and speed to detect railway lines.

2) A rapid railway line detection algorithm is proposed that utilises keypoint alignment to perform track detection in simple scenarios. Through analyzing the track shapes, we construct a track basic shape library predicated on keypoints, selecting the track shape with the highest confidence score as the detection outcome.

3) A track positioning module (CTPM) is proposed to detect tracks in complex scenes such as curved tracks, tunnels and turnouts. Track detection accuracy is further improved by using the track anchor feature pooling module. And a new loss function ( $L_{PIOU}$ ) for track detection is introduced, which better reflects the influence of perspective effect.

4) After treating the detection results for the turnout scene, the current track position of the train can be extracted. By pairing the track lines in the image, and specializing the turnout in different scenes, the interference tracks lines are suppressed.

The structure of the remainder of this article is as follows. Section II describes the flow of the algorithm and discusses the keypoint alignment, the CTPM and turnout post-treatment algorithm, respectively. Section III mainly introduces the datasets and related experimental results. These are crucial, as they provide the evidence needed to validate the effectiveness of the algorithm. Section IV concludes with an analysis of the limitations and briefly outlines future work.

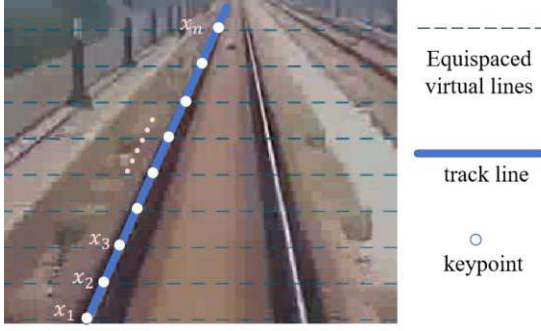


Fig. 2. The track keypoints.

## II. PROPOSED METHODS

### A. Representation of the Track Keypoints

A typical rail-track consists of two slender, parallel rails characterized by uniform and specific shapes and curvatures. Detection precision can be significantly improved by taking advantage of prior knowledge of track shapes. Standard object detection algorithms employ rectangular bounding boxes to represent detected objects. However, rectangular boxes do not reflect the shape of the track well. To reflect the position and shape of the track at the same time, the representation based on keypoints is integrated into the track detection.

The specific operation is to conceptualize the track as an array of sampling points. This representation divides the designated part of the image excluding the sky, into equal intervals and identifies the keypoints as the intersections of the track line with these intervals, as shown in Fig. 2. Whether it is a straight track or a curved track, as long as the positions of these points can be detected, the position and shape of the track can be identified.

### B. Keypoint Alignment

1) *Analysis of Track Geometry*: The basic shapes of railway track include three types, straight tracks, circular tracks and transition curve tracks. For straight tracks, the centripetal force is zero, for circular tracks, the centripetal force is related to speed. If straight tracks are connected to circular tracks directly, the centripetal force changes sharply, which can affect the safety of the train running. Therefore, it is crucial to select the appropriate transition curves.

Considering the simplicity of the cubic parabolic transition curve, the convenience of design and maintenance, and the extensive experience in field application and maintenance, dedicated passenger rail lines in China prefer the cubic parabolic transition curve as the primary choice for transition curves. As shown in Fig. 3,  $x$  and  $F(x)$  denote the horizontal and vertical coordinates of the reference frame, respectively.

The formula for the track at non-railway turnout is as follows.

$$F(x) = \begin{cases} 0 & x \in (-\infty, 0) \\ \frac{x^3}{6Rl_0} & x \in [0, l_0] \\ y_{center} - \sqrt{R^2 - (x - x_{center})^2} & x \in (l_0, l_j) \end{cases} \quad (1)$$

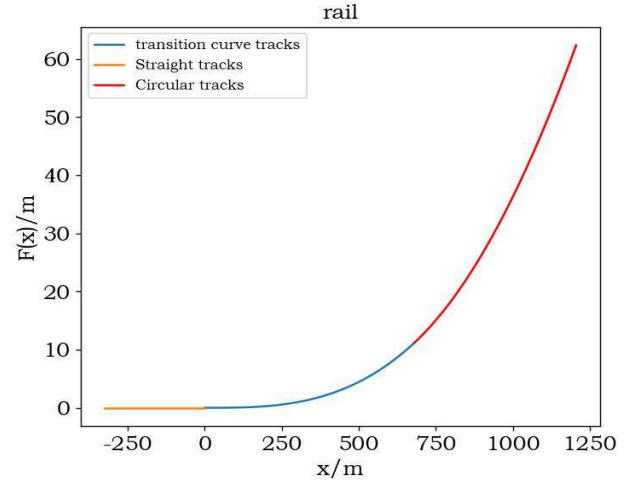


Fig. 3. The shape of railway curved track under the curve radius of 7000m in Beijing-Shanghai high-speed railway.

where

$$x_{center} = l_0 - \frac{F'(l_0) * R}{\sqrt{1 + F'(l_0)^2}} \quad (2)$$

$$y_{center} = F(l_0) + \frac{R}{\sqrt{1 + F'(l_0)^2}} \quad (3)$$

$$F'(l_0) = \frac{l_0}{2R} \quad (4)$$

where  $R$  denotes the radius of the curve corresponding to the circular tracks.  $l_0$  denotes the length of transition curve, which is selected according to the *Code for Design of High Speed Railway (TB10621-2014)* [27].  $l_j$  denotes junction of circular track and the next transition curve track.  $x_{center}$  and  $y_{center}$  denote the horizontal and vertical coordinates of the center of the circular track respectively.

In this scenario, when the train runs to different positions, the track in the front-view camera shooting range (assuming that it is 325m away from the camera) can be calculated, as shown in the following Fig. 4 and Fig. 5. When the train runs to  $(x_0, F(x_0))$ , the track offset with a forward distance of  $D$  can be calculated by

$$g(D) = F\left(x_0 + \frac{D}{\sqrt{1 + F'(x_0)^2}}\right) - \left(F(x_0) + \frac{F'(x_0) * D}{\sqrt{1 + F'(x_0)^2}}\right) \quad (5)$$

2) *Camera Offset Compensation in Curved Tracks*: If the keypoints of the curve tracks obtained by direct calculation are mapped to the templates, it is found that there are tiny deviations. This is because during the running of the train on curved tracks, the train leans toward the side, which leads to a relative displacement between the camera and the actual track. Therefore, the coordinates of the curved track offset need to be compensated.

When the train runs on curved tracks, the longitudinal deviation of the camera position is very small and can be ignored. The lateral deviation of the camera is large and needs to be compensated. The compensation value  $\Delta x$  is calculated by

$$\Delta x = H * \tan \gamma \quad (6)$$

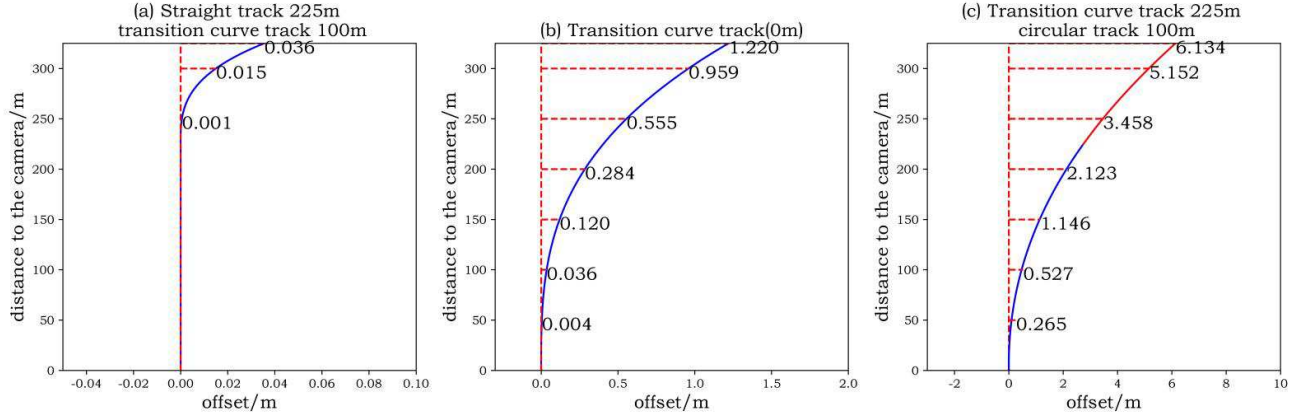


Fig. 4. The track offset of the train running from the straight track to the curved track. (a) Straight track 225m transition curve track 100m. (b) Transition curve track, 0m away from the junction between the straight track and the curved track. (c) Transition curve track 225m circular track 100m.

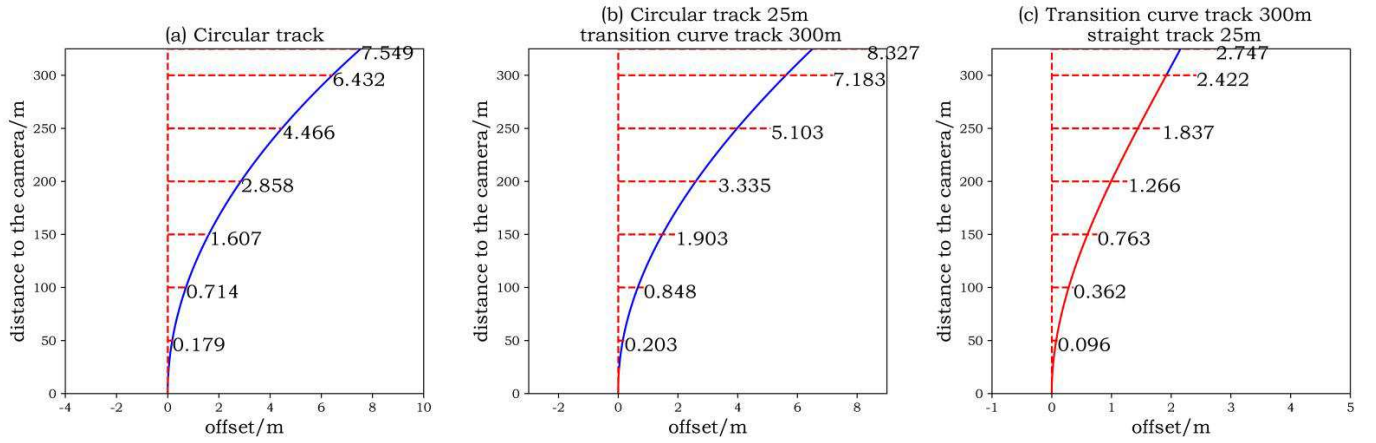


Fig. 5. The track offset of the train running from the curved track to the straight track. (a) Circular track. (b) Circular track 225m transition curve track 100m. (c) Transition curve track 300m straight track 25m.

where  $H$  denotes the height of the camera,  $\gamma$  denotes the inclination angle of the train. The resultant force of gravity and rail support acts as a centripetal force to avoid derailment.  $\tan \gamma$  can be calculated by

$$mg \tan \gamma = m \frac{v^2}{r} \quad (7)$$

then

$$\Delta x = H * \frac{v^2}{gr} \quad (8)$$

where  $m$  denotes the quality of the train,  $g$  denotes acceleration of gravity,  $r$  denotes track radius,  $v$  denotes design speed.

3) *Keypoints Alignment Method*: Given the focal length of the camera  $f$ , the position of the track in the camera coordinate system, and the offset compensation of the camera, the specific shape of the track on the image can be calculated by conversion of the camera coordinate system and the pixel coordinate system [28], [29], [30], [31], [32]. As shown in Fig. 6 and Fig. 7, that is, the calculated shape of the track on the image.

All track shapes are calculated in advance and saved in the track basic shape library, which is used as the follow-up detection templates. In this paper, an index *KNCC* (*Keypoint*

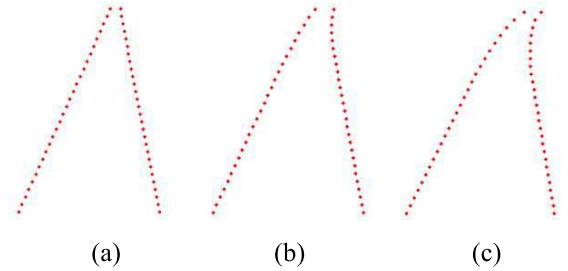


Fig. 6. The template of the train running from straight tracks to curved tracks. (a) Straight track 225m transition curve track 100m. (b) Transition curve track, 0m away from the junction between the straight track and the curved track. (c) Transition curve track 225m circular track 100m.

*Normalized Cross Correlation*) for track shape matching is defined by

$$KNCC = \frac{\sum_{x,y} (t(x,y) * a(x,y))}{\sqrt{\sum_{x,y} t(x,y)^2 * \sum_{x,y} a(x,y)^2}} \quad (9)$$

where  $x$  and  $y$  denote the horizontal and vertical coordinates corresponding to the calculated keypoints of the template,  $t(x, y)$  denotes the pixel value of the template at this position,  $a(x, y)$  denotes the pixel value of the input image at this position.

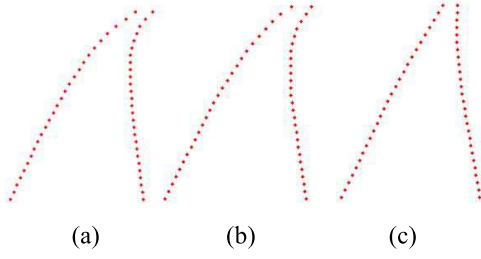


Fig. 7. The template of the train running from curved tracks to straight tracks. (a) Circular track. (b) Circular track 225m transition curve track 100m. (c) Transition curve track 300m straight track 25m.

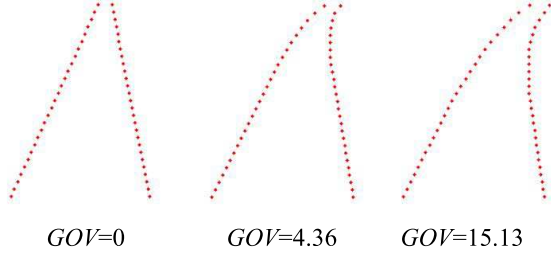


Fig. 8. The  $GOV$  corresponding to different track shapes from left to right are 0, 4.36 and 15.13, respectively.

The pixel value of the keypoint of the template is the manually selected pixel value of a certain rail surface. Different from the traditional template matching algorithm for calculating the matching degree of all pixels on the target image of the template image [33] [34] [35], this method only calculates the pixel matching degree at the keypoint position, and fewer calculation points and pre-calculated positions improve the calculation speed of the algorithm.  $KNCC$  measures the cosine distance between the template keypoint vector and the image keypoint vector. This standardization process effectively eliminates the influence of overall strength change. Even if the pixel values of the whole image fluctuate due to illumination changes, the relative relationship between the pixel values at keypoints is preserved, thus allowing accurate matching. Unlike the traditional template matching algorithm, which considers all pixels in the target image,  $KNCC$  only calculates the degree of pixel matching of keypoints. This reduces the influence of illumination changes in non-key areas and speeds up the calculation of the algorithm. A threshold is set to compare the matching degree of all track shapes with the input track, and the track shape with the highest  $KNCC$  and exceeding the threshold is used as the final track detection result.

It is highly time-consuming to traverse all track shapes in the library. This approach does not capitalize on the correlation between the previous and current frame images, devotes the majority of detection time to matching track shapes that are distant from the actual track position. Consequently, we propose a systematic and rapid track shape detection algorithm. Each search for the best track shape does not traverse all track templates, and only traverses track templates similar to the previous frame image according to the  $GOV$  (global offset value) of each shape.  $GOV$  is defined by

$$GOV(temp^r) = \frac{\sum_h (x_h^r - x_h^s)}{Len} \quad (10)$$

TABLE I  
COMPARISON OF TIME COMPLEXITY OF DIFFERENT SEARCH METHODS

| Layer name | Output size | CTPM ResNet18                               | CTPM ResNet34         |
|------------|-------------|---------------------------------------------|-----------------------|
| block1     | 192×320     | 3×3, stride=2, 3<br>3×3, max pool, stride=2 |                       |
| block2     | 96×160      | 3×3 residual block, 2                       | 3×3 residual block, 3 |
| block3     | 48×80       | 3×3 residual block, 2                       | 3×3 residual block, 4 |
| block4     | 24×40       | 3×3 residual block, 2                       | 3×3 residual block, 6 |
| block5     | 12×20       | 3×3 residual block, 2                       | 3×3 residual block, 3 |
| block6     |             | 1×1 conv<br>feature pooling                 |                       |
|            |             | 2 parallel fully connected layers           |                       |
| Params.    |             | 10.95 M                                     | 21.35 M               |

where  $temp^r$  denotes track template to be calculated,  $x_h^r$  denotes the abscissa of the track template to be calculated at the height  $h$ ,  $x_h^s$  denotes the abscissa of the standard straight rail at height,  $Len$  denotes the number of keypoints of  $temp^r$ . It can be seen from Fig. 8. that  $GOV$  can represent the bending degree well and overall position of different track forms, and then represent its shape.

All the tracks in the library are arranged in order and numbered marked as  $\{T_1, CT_2, CT_3, \dots, CT_N\}$ . The more adjacent the track shape is, the higher the similarity is. The maximum number of searches  $num$  is defined without affecting the speed and precision, the search range of the current frame image is  $\{T_{i-num/2}, T_{i+1-num/2}, \dots, T_i, \dots, T_{i-1+num/2}, T_{i+num/2}\}$  ( $T_i$  is the matching track shape of the previous frame). After the corresponding track shape is retrieved in this range, the retrieval interval of the next frame is updated according to the position of the shape. Because the sorting process is completed before the track detection, the overall detection speed is fast.

Currently, the high-speed train runs at a speed of 350km/h (97.2m/s). The camera takes 25 images per second, which means that the train moves 3.89 meters per frame. This distance response is very small in the image. Even when turning sharply, the track shape gradually changes from frame to frame instead of suddenly changing, and there is a high correlation between adjacent frames.  $GOV$  takes advantage of this continuity. When searching only for the track template similar to the previous frame, it can quickly adapt to change in track shape while maintaining high detection speed and accuracy.

Although the keypoint alignment method has great advantages for simple scenes, it performs not well in complex scenes, such as turnouts. Therefore, it is necessary to design a deep learning module for track detection to enhance performance in the scenes.

### C. Complex Track Positioning Module (CTPM)

1) *Overall Structure of the Module:* The proposed CTPM consists of the backbone and the feature pooling module. As an important part of feature extraction, the backbone can be replaced by other CNN flexibly. In this paper, the backbone

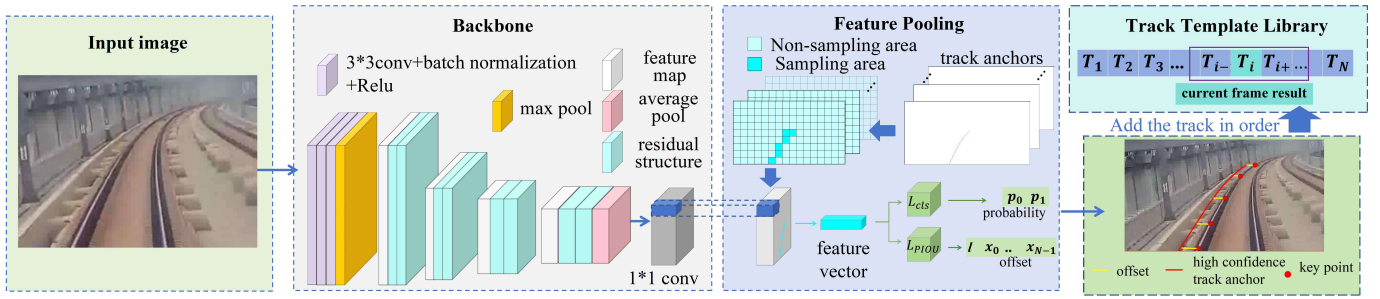


Fig. 9. Complex Track Positioning Module (CTPM). For any input image, the backbone extracts its features. Then the CTPM employs the anchor track preset to perform feature pooling and calculate the feature vector. Finally, the loss of classification and regression is calculated respectively. At the same time, by posting-treatment of railway turnout, the accurate track position is obtained.

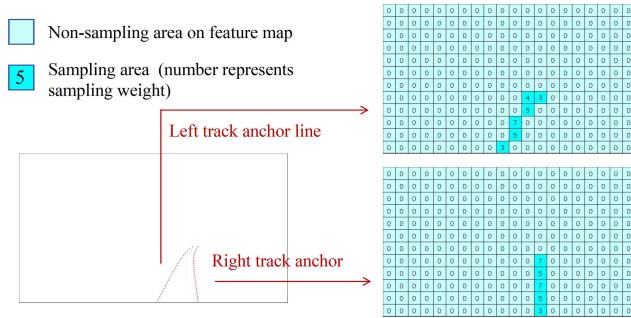


Fig. 10. Calculation process of characteristic pool of track anchor line. (The zero-free region covered by the track anchor line, which is pooled, and the feature vector is calculated for further calculation. The numbers in the image represent the weight of each part when calculating the feature vector. The other area represents the feature map area that is not pooled for calculation.)

used is a variant of ResNet18 and ResNet34 [36]. To improve detection speed, this paper fine-tunes the original ResNet [36], and replaces the  $7 \times 7$  convolution at the input with three  $3 \times 3$  convolutions. The network structure is shown in TABLE I.

In this paper, anchors are virtual lines on the image plane, which is composed of part of the track shape calculated in the previous section. The feature vector after feature pooling becomes the basis for predicting the track. Finally, it is connected with two parallel fully connected layers, and the anchors are classified and the keypoint offsets are regressed, respectively, so as to predict the final output track. Meanwhile, the GOV of the CTPM detection results should be calculated, and the basic shape library of the track should be supplemented in order for the subsequent detection of the track images.

For the first image of a new group of consecutive frames, it is difficult to detect it with the keypoint alignment method, so we can only detect the result with the CTPM and insert it into the library. And start with this, and carry out the cycle of GOV sort search. The overall process of this part is shown in Fig. 9.

2) *Feature Pooling Based on Track Anchor Line*: The points on the anchor line determine the region on the feature map  $F$  used to calculate the probability and offset of the tracks. Since the anchor line is set according to the track shape analyzed in the previous section, the attention of the model is projected in the track approximately, which is also the key to ensuring the high accuracy of the model. The calculation

process of the track anchor lines is shown in Fig. 10. This step splits each line in the track template library into two anchor lines preset on the feature map. The formula is calculated by

$$x_f = \frac{x_o}{W_o} * W_f \quad (11)$$

$$y_f = \frac{H_o - y_o}{H_o} * H_f \quad (12)$$

where  $x_o$  and  $y_o$  denote the track form in the keypoint coordinates with the image (Opencv coordinate system),  $x_f$  and  $y_f$  represent the position of keypoints on the feature map,  $W_f$  and  $H_f$  denotes the width and height of the feature map.

When the position coordinates of the anchor line on the feature map are calculated, it is inevitable that there are multiple keypoints of track shape corresponding to the same feature map position. Therefore, it is necessary not only to calculate the anchor line position but also to record the number of occurrences of each position, which denotes the weight. After normalising the number of occurrences of each position, a weight matrix is obtained with the same two-dimensional size as the feature map. The formula is calculated by

$$we_i = \frac{n_i}{\sum_{j=1}^N n_j} \quad (13)$$

where  $we_i$  is the normalized weight corresponding to position  $i$ , the number of occurrences of a certain position  $i$  is  $n_i$ , and the total number of positions is  $N$ . The weight matrix weights the corresponding regions of the feature map to calculate the feature vector. The formula is calculated by

$$\text{FeatureVector} = \sum_{i=1}^N we_i * F_i \quad (14)$$

The feature vector is computed by element-wise multiplying the feature map  $F$  with the weight matrix  $we$  and summing the products. This feature vector is fed into two parallel fully connected layers to compute the corresponding probabilities and offset regressions, enabling track classification and precise keypoint localization.

The weight matrix is generated strictly on the basis of geometric prior knowledge of the track. The positions of track anchors are preset from the mathematical modeling of track shapes, and their projections on the image plane directly correspond to the actual spatial distribution of the track. The weight  $we_i$  at each position in the weight matrix is determined

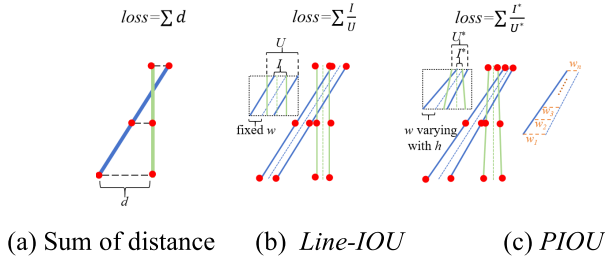


Fig. 11. Different loss to measure the similarity between predicted tracks and GT.

by the sampling frequency of anchors on the feature map, essentially reflecting the spatial distribution density of the track in that region.

3) *Distance Loss and Classification Loss*: The loss function is used to quantify the difference between the predicted value and the GT (ground truth) of the model. It receives the predicted value and GT of the model as input, and then calculates a numerical value, which represents the difference between the predicted value and GT. The smaller the loss, the closer the predicted result to GT and the better the fit of the model. The loss function in this paper consists of two parts, namely, classification probability loss  $L_{cls}$  and perspective track IOU loss  $L_{PIOU}$ . The overall loss function is

$$L(\{p_i, x_i\}_{i=0}^N) = \lambda \sum_{i=0}^{N-1} L_{cls}(p_i, p_i^*) + L_{PIOU}(x_i, x_i^*) \quad (15)$$

where  $\lambda$  denotes coefficient of balance,  $p_i$  denotes classified the output of anchor lines corresponding to tracks,  $p_i^*$  denotes classification target of anchor line corresponding to track,  $x_i$  denotes regression output of track keypoints,  $x_i^*$  denotes regression target of track keypoints, classified loss  $L_{cls}$  denotes focal loss [22], [37].

As shown in Fig. 11, the existing methods often use the distance loss between the keypoint and GT (smooth L1) or the Line-IOU with a fixed width as the similarity function to measure the predicted track line and GT. However, using distance as a loss to make the keypoints on a track independent of each other does not take into account the relationship between the points as a whole. Line-IOU regresses the track line as a whole, but does not consider the influence of the perspective principle on the tracks at different positions. In this paper, the IOU loss of perspective track (*PIOU*) is proposed. It is defined by

$$L_{PIOU} = 1 - \frac{\sum_{i=0}^{N-1} I_i}{\sum_{i=0}^{N-1} U_i} \quad (16)$$

where

$$I_i = \min(x_i + w_i, x_i^* + w_i) - \max(x_i - w_i, x_i^* - w_i) \quad (17)$$

$$U_i = \max(x_i + w_i, x_i^* + w_i) - \min(x_i - w_i, x_i^* - w_i) \quad (18)$$

$$w_i = \frac{w_t}{2} = 0.01h_i - 3.1 \quad (19)$$

where  $w_t$  denotes the rail surface width (The calculation process is in The adaptive allowable errors),  $h_i$  denotes the ordinate of the keypoints. In order to prevent the PIOU from



(a) Running in curved tracks      (b) Running in straight tracks

Fig. 12. Convergence scene.

being calculated as a negative number, if  $\min(x_i + w_i, x_i^* + w_i) - \max(x_i - w_i, x_i^* - w_i) < 0$ , that is

$$I_i = 0, U_i = \max(x_i + w_i, x_i^* + w_i) - \min(x_i - w_i, x_i^* - w_i) \quad (20)$$

If predictive keypoints have no corresponding GT, that is

$$I_i = 0, U_i = 0 \quad (21)$$

The *PIOU* in this paper is designed based on the image perspective projection, a common feature of all track types. Additionally, the width parameter in the *PIOU* calculations refers to the width of the rail surface, not the track gauge. Therefore, when the width of the rail surface is known in advance, the loss based on *PIOU* is suitable for all kinds of rail types. The pseudocode flow of CTPM is as follows.

#### D. Posting-Treatment of Railway Turnout

Turnouts are essential components of railway tracks that allow locomotives to change from one track to another. The safety and reliability of the turnout are crucial for the uninterrupted operation of the train [38], [39], [40]. There are various types of turnout, including the single turnout, the double turnout, the crossover turnout, etc. Among these, the single type is the most commonly used type, accounting for more than 90% of all turnouts in China. It can divide one track into two, with the main line being straight and the side line diverging to the left or right. Single turnout is favoured for its simple structure, ease of operation, and strong adaptability and is widely used at railway stations, track sections, and other locations. As the basic work of high-speed rail track detection, this study chooses single turnout as the starting point, aiming at solving the core problem of current running track identification first, and laying a methodological foundation for the subsequent research on complex turnout.

The challenges in detecting tracks for turnouts are evident in two aspects. First, the distributions are complex and different in shape. Due to the diversity of turnout design, there are significant differences in structure, size and shape. It is impossible to analyze every turnout pattern. Second, another challenge pertains to the training of deep learning models. Due to the intricate track environment, the deep learning model is disrupted by other tracks during the training process, leading to the inability to determine the exact track on which the train is located. To address these issues, this paper introduces an innovative post-treatment method for detecting the single turnout.

**Algorithm 1** Complex Track Positioning Module

---

**Input:**

- Feature map  $F$  from backbone network
- Precomputed track anchor lines  $A$

**Output:**

- Final track keypoints  $K$

```

1: function CTPM ( $F, A$ )
2:   // 1. Feature Pooling
3:   anchor_scores ← [], pooled_features ← []
4:   for each  $roi \in A$  do
5:     feature ← FeaturePooling( $F, roi$ )
6:     pooled_features.append(feature)
7:     score ← ClassifyAnchorAsForeground(feature)
8:     anchor_scores.append(score)
9:   end for
10:  // 2. Select  $N$  anchor lines with the highest score
11:  top_anchor_indices ← SelectTopKIndices (anchor_scores,  $N$ )
12:  // 3. Keypoints regression
13:  for each keypoint  $\in$  top_anchor_indices do
14:    // Optimization of keypoints position using PIOU
15:     $K \leftarrow$  RefineKeypoints(keypoint, pooled_feature, PIOULoss)
16:  end for
17:  return  $K$ 
18: end function

```

---



Fig. 13. Divergence scene.

1) *Scene Division*: The scene captured by the front-view camera of the train is divided into two types, convergence scene and divergence scene. The convergence scene refers to the track where a pair of curved tracks and a pair of straight tracks merge into a single pair of straight tracks, as shown in Fig. 12. The divergence scene refers to the track where a pair of straight tracks diverges into a pair of curved tracks and a pair of straight tracks, as shown in Fig. 13. To facilitate the discussion, the first step of the post-treatment method is to divide the scene.

The tracks detected by the CTPM are saved in the form of  $4 \{x_1, x_2, \dots, x_n\}$ .  $x_i$  is track abscissa at  $h = 710$  in images. The four  $x_n$  are sorted from small to large, and then the smaller two are subtracted, and the larger two are subtracted. If these two differences are less than 8 (the width of the rail surface under this  $h$ ), it indicates that the starting points of the four tracks are derived from a pair of tracks and belong to the divergence scene. On the contrary, it belongs to the convergence scene.

2) *Tracks Pairing*: The retained track is a pair of tracks currently running on the train. The detection result of the CTPM is four unrelated tracks, so the tracks need to be

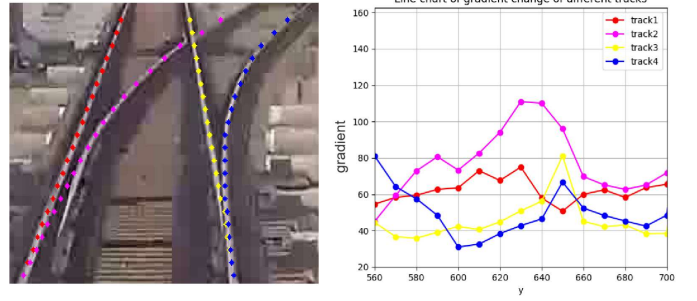


Fig. 14. Line chart of gradient change of different tracks. (The keypoints at the bottom of track2 are located in the gap, and the gradient is obviously larger than other tracks.)

paired. There are only two categories of tracks appearing on the image, straight and non-straight. Therefore, the two tracks belonging to the straight track can be paired directly, and the other two can be paired. To determine whether the track is a straight track, this paper adopts a detection method based on slope comparison. Calculate the slopes at the beginning and end of the track, and determine whether the two slopes are the same (or whether the difference between the two slopes is less than a preset threshold).

$$K_s = \frac{x_{sn} - x_s}{\Delta h}, K_e = \frac{x_e - x_{ep}}{\Delta h} \quad (22)$$

where  $K_s$ ,  $K_e$  denotes the slope,  $x_s$ ,  $x_{sn}$  represent the abscissa of the two keypoints closest to the camera,  $x_e, x_{ep}$  denotes the abscissa of the two keypoints that are farthest from the camera,  $\Delta h$  denotes the difference between the ordinates of adjacent keypoints.

$$|K_s - K_e| < \tau_K \quad (23)$$

where  $\tau_K$  is the threshold, if the slope difference between the beginning and the end is less than the threshold, then this track is considered to be a straight track. Otherwise, it is judged as a non-straight track.

3) *Turnout Post-Treatment for Convergence Scene*: The turnout in the convergence scene can be further divided into two scenarios. The curved rail merges with the straight rail and the straight rail is driven to the straight rail. The basic shape of the straight rail running to the straight rail is the same as the straight rail in the keypoint alignment method. Therefore, it is only necessary to judge whether the straight rail track pair in the convergence scene is consistent with the straight rail match of the keypoint alignment method. If it is consistent, the straight rail match is retained as the final output. If it is inconsistent, it shows that the current scene belongs to the case where the curved rail is merged into the straight rail, and another pair of non-straight rail matches is retained as the output result.

4) *Turnout Post-Treatment for Divergence Scene*: The locomotive is guided by the switch rail when entering and leaving the turnout, which is the main part of the switch. Take the right turn turnout as an example. When the left switch rail is far away from the left straight rail, the train goes straight. When the right switch rail is far away from the right straight rail, the train turns right. The distance from the switch rail to the straight rail leads to a *gap* in the track corresponding

to the non-driving space of the train, as shown in Fig. 14. In tracks, these gaps appear as a kind of discontinuity. It causes a sudden change in the track pixel value. By capturing the sudden change of this pixel value, the falsely detected track pairs can be identified and suppressed effectively. Leveraging these structural differences, this paper proposes a gradient-based method to enhance the reliability of track detection in divergence scenes. The core idea is to evaluate the structural integrity of detected tracks by analyzing the gradient magnitude of local image regions around key points.

Based on the given track coordinate data, for each key point of each track, the track direction  $\theta$  formed by its connection to the next point is first determined. For the direction angle  $\theta$ , a 3\*3 convolution kernel  $KN$  is designed.

$$KN = \begin{bmatrix} \sin \theta - \cos \theta & \sin \theta & \sin \theta + \cos \theta \\ -\cos \theta & 0 & \cos \theta \\ -\sin \theta - \cos \theta & -\sin \theta & -\sin \theta + \cos \theta \end{bmatrix} \quad (24)$$

The gradient of the point  $(x, y)$  in the direction  $\theta$  can be defined as

$$G_\theta(x, y) = \sum_{u=-1}^1 \sum_{v=-1}^1 I(x+u, y+v) * KN(1+u, 1+v) \quad (25)$$

The average value of the area around the keypoint is taken as the feature of the point. This feature reflects the change of angular gradient along the direction  $\theta$  in the neighborhood of the keypoint as shown in Fig. 14. If all the keypoints are located on the rail, the gradient value changes little. On the contrary, it means that some points are located in a *gap*. Finally, the correct tracks can be reserved.

### III. EXPERIMENTATION

#### A. Datasets

In order to analyze the proposed method quantitatively, this paper takes the rail where the train runs in the high-speed railway scene with a speed of 350km/h as the research object. The front image of the train is captured by the front camera of the train, and high-speed railway track datasets (HRTD) are established. The datasets are collected from the Beijing-Shanghai high-speed railway with a resolution of  $720 \times 1280$ . The datasets contain a wealth of representative scenarios, including tunnels, uphill and downhill and turnouts. The datasets provide 7189 images with high-quality annotations, which are divided into train sets and test sets. The ratio of train sets to test sets is 4:1. In addition, this study further divides the datasets, including 3680 straight rail images and 3509 non-straight rail images (including curved rail and turnout scenarios).

To verify the robustness of the algorithm in a complex natural environment (rain or snow weather, different lighting conditions), this paper enhances the original datasets, uses *imgaug* library to add rain or snow noise to each picture, and randomly changes the brightness of the picture to simulate different lighting conditions. In some scenes, obstacles are added to cover the situation. Finally, we form complex weather track datasets (CWTD) with the same size and proportion.

When annotating the images, although the task is to detect the current track of the train, it is still necessary to annotate all the tracks in front. In addition, in order to ensure that the keypoints of the annotation are located on the center line of the rail surface, the actual manual annotation should be on the left and right boundaries of the rail surface. The midpoints are calculated as the keypoints of GT.

#### B. Experimental Parameter Settings

The minimum curve radius of the general section of the Beijing-Shanghai high-speed railway is 9000 meters to 12000 meters, but it can be reduced to 7000 meters under difficult conditions. Therefore, in the keypoint alignment part of this experiment, a total of 11 scenarios (with a span of 500 meters) are designed for the track shape corresponding to the curve radius of 7000 meters to 12000 meters. In each scenario, 15 track forms are designed in the order of straight tracks to transition curve tracks, transition curve tracks, transition curve tracks to circular tracks, circular tracks to transition curve tracks, transition curve tracks to straight tracks, with a total of 165 track templates.

The GOV search range number is 18. The KNCC matching threshold is 0.7, and the slope threshold  $\tau_k$  is 0.05. Allowable error According to the rail width formula, when  $h=710$  pixels, the allowable error is 4 pixels.  $H=460$  pixels is 1 pixel. When more than 85% of the keypoint of the track line are detected correctly, it is determined that the line detection is successful. The deep learning experiments are all carried out on NVIDIA RTX A5000. In the experiment, each track shape is split into two anchor lines preset on the feature map and stored in advance.

#### C. Evaluating Indicator

1) *The Adaptive Allowable Errors*: It is very important to ensure the accuracy of the detection results when performing a track inspection. Since the track is a major component of railway traffic, any minor deviation can have a significant impact on the safe operation of the train. Therefore, during track detection, it is necessary to ensure that the coordinates of the detected keypoints are strictly within the effective range of the rail surface, that is, these keypoints should be located between the left and right boundaries of the rail surface. In this paper, the keypoints are annotated at the center of the left boundary and the right boundary of the rail surface. The final allowable error is half the width of the rail. In the case of a straight track, the left and right boundaries of a pair of tracks are four straight lines converging to the center. The abscissa  $x$  and ordinate  $h$  of the keypoints on the straight line should satisfy the linear relationship. By subtracting the analytical expressions of the left and right boundaries of the same track, the track width at different heights can be obtained. The analytical formula of the four straight lines from left to right can be calculated by

$$\begin{cases} x_1 = -0.448h + 961.08 \\ x_2 = -0.428h + 954.88 \\ x_3 = 0.184h + 702.36 \\ x_4 = 0.204h + 696.19 \end{cases} \quad (26)$$

The relationship between the allowable error of tracks  $\delta_{left}, \delta_{right}$  and the ordinate  $h$  can be calculated by

$$\begin{cases} \delta_{left} = \frac{x_2 - x_1}{2} = 0.01h - 3.1 \\ \delta_{right} = \frac{x_4 - x_3}{2} = 0.01h - 3.09 \end{cases} \quad (27)$$

The existing research on track detection adopts the error evaluation of key points based on Euclidean distance or the fixed-width Line-IOU, without considering the perspective characteristics of *near objects appearing larger and far ones smaller* in railway scenes, which is inconsistent with the demand of *higher precision of far track* in practical engineering. Finally, when  $h = 710$ , the allowable error on the left and right tracks is 4 pixels. When  $h = 460$ , the allowable error of the left and right tracks is about 1 pixels. At different heights  $h$ , the same pixel error may lead to different actual errors. This paper has higher precision requirements for the detection of far tracks than near tracks.

2) *Metrics of Efficiency*: FPS(frames per second) is a key performance index to measure the image processing speed of the algorithm. It specifies how many frames the algorithm can process and generate in a specific time. This metric is calculated by taking into account a single image per batch and a fixed amount of input, thus ensuring that it evaluates the processing power of the model itself, rather than the impact of I/O operations. In view of this, this study selected FPS as the core standard to measure processing efficiency.

3) *Metrics of Precision*: In this paper, three evaluation metrics are defined, which are  $ACC_{point}$ ,  $MR_{point}$  and  $ACC_{line}$ .  $ACC_{line}$  is defined by

$$ACC_{point} = \frac{\sum_{i=0}^M C_i}{\sum_{i=0}^M T_i} \quad (28)$$

where  $M$  denotes The total number of images in the dataset,  $T_i$  denotes The total number of keypoints of the track in the  $i_{th}$  image,  $C_i$  denotes The number of correct keypoints. The keypoints calculated are only within the allowable error range of GT, and are considered to be detected correctly.  $MR_{point}$  is defined by

$$MR_{point} = 1 - \frac{\sum_{i=0}^M P_i}{\sum_{i=0}^M T_i} \quad (29)$$

where  $P_i$  denotes the total number of track keypoints detected in the image.  $ACC_{line}$  is defined by

$$ACC_{line} = \frac{\sum_{i=0}^M S_i}{2M} \quad (30)$$

where  $S_i$  denotes the number of correct detected track lines. Only when 85 % (and above) of the keypoints on the track line are detected correctly, it is considered that the line detection is correct.

#### D. Experimental Result

In TABLE II, Track<sup>2</sup>Net is compared with different methods on HRTD, including Canny, Dynamic programming, HOG, LaneATT [22], CLNet [25] and GANet [26]. This experiment conducted a comparative analysis of model accuracy and

TABLE II  
COMPARISON OF MODELS ON HRTD

| Model                             | $ACC_{point}$ (%) | $ACC_{line}$ (%) | $MR_{point}$ (%) | FPS        |
|-----------------------------------|-------------------|------------------|------------------|------------|
| Canny                             | 50.71             | 56.58            | 38.16            | 127        |
| Dynamic programming               | 62.94             | 68.43            | 30.26            | 0.4        |
| HOG                               | 71.97             | 75.67            | 10.33            | 10         |
| LaneATT (ResNet18)                | 85.87             | 87.67            | 2.01             | 144        |
| LaneATT (ResNet34)                | 87.04             | 88.43            | 1.92             | 101        |
| CLNet (ResNet18)                  | 86.94             | 87.97            | 1.78             | 115        |
| CLNet (ResNet34)                  | <u>89.03</u>      | 89.16            | 1.23             | 81         |
| GANet (ResNet18)                  | 86.95             | 89.15            | 1.45             | 109        |
| GANet (ResNet34)                  | 87.87             | <u>90.29</u>     | <u>1.12</u>      | 82         |
| Track <sup>2</sup> Net (ResNet18) | 88.33             | 88.08            | 1.34             | <b>243</b> |
| Track <sup>2</sup> Net (ResNet34) | <b>90.95</b>      | <b>91.48</b>     | <b>0.96</b>      | <u>236</u> |

execution time, where each evaluation metric is highlighted with boldface for the best performance and underscored for the second. Among them, the canny, dynamic programming and HOG belong to traditional image processing, and the other methods are based on deep learning. Canny has a good detection speed of 127 FPS, but its accuracy is not good. In contrast, the dynamic programming method has a higher detection accuracy, but it has a large amount of calculations and a low speed. The traditional HOG is mostly used for target detection, and the output result is not the track lines required by this task. Therefore, the HOG compared in this paper has been adjusted. The specific operation is as follows; the gradient direction histogram of statistical local area is obtained through the feature extraction of HOG. Starting from the pixels at the bottom of the image at regular intervals, the path is searched upward, and the direction with the largest gradient response is found as the extension direction of the path. If the length of the path exceeds a certain threshold, it is considered as an effective track line. In Table II HOG has further improved its accuracy, but its speed is not high. Unlike traditional image processing, the deep learning method has better detection accuracy and speed.

Using ResNet34 as the backbone model, Track<sup>2</sup>Net is 1.92 % higher than CLNet in  $ACC_{point}$ , 1.19 % higher than GANet in  $ACC_{line}$ , and 0.16 % lower than GANet in  $MR_{point}$ . While ensuring that the accuracy is improved, the detection speed of the Track<sup>2</sup>Net model is much faster than other models. The anchor lines preset by LaneATT on the image are all straight lines, which cannot be fitted in the face of some special track match. GANet transforms the track into the offset between other keypoints and the starting point, and it is easy to miss some track lines. The detection of CLNet at the turnout

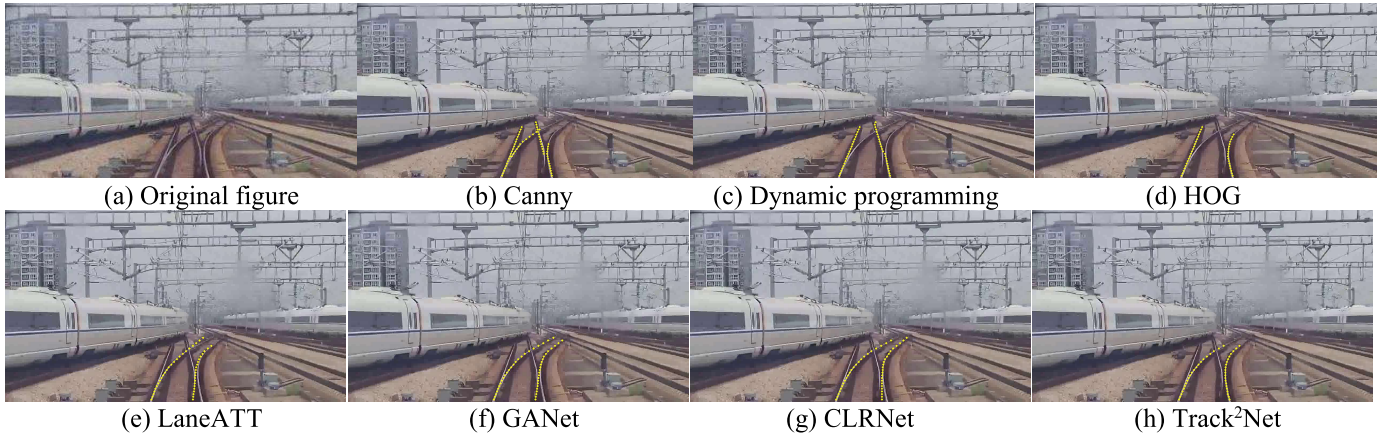


Fig. 15. Detection results of different models.

TABLE III  
COMPARISON OF MODELS ON CWTD

| <i>Model</i>     | $ACC_{point}$ (%) | $ACC_{line}$ (%) | $MR_{point}$ (%) | <i>FPS</i> |
|------------------|-------------------|------------------|------------------|------------|
| LaneATT          | 83.66             | 85.07            | 4.36             | <u>101</u> |
| CLRNNet          | <u>85.84</u>      | <u>88.64</u>     | <u>2.61</u>      | 81         |
| GANet            | 84.23             | 86.79            | 2.87             | 82         |
| <b>Track²Net</b> | <b>87.76</b>      | <b>90.61</b>     | <b>1.19</b>      | <b>171</b> |

TABLE IV  
COMPARISON OF TIME COMPLEXITY OF DIFFERENT  
SEARCH METHODS

| <i>method</i>      | <i>Time Complexity</i> | <i>FPS</i> |
|--------------------|------------------------|------------|
| GOV sort search    | O (num)                | 322        |
| Enumeration search | O (n)                  | 30         |

is interfered with by other tracks. On the contrary, the method in this paper can detect the current train track, and the FPS reaches the fastest. The detection results of different models at turnout are shown in Fig. 15.

Table III serves as a complement to Table II, focussing on assessing the robustness of each model in CWTD. Resnet34, the backbone network with higher accuracy, is selected for the experiment. Notably, GANet experiences the most significant impact, with  $ACC_{point}$  decreasing by 3.64 and  $ACC_{line}$  dropping by 3.5. On the contrary, Track²Net shows the least decline, with  $ACC_{point}$  dropping by 3.19 and  $ACC_{line}$  by 0.87, but still retains the highest accuracy and speed. This evidence underscores the superior robustness of Track²Net among the models evaluated.

TABLE IV compares the effects of GOV sort search and Enumeration search on the detection speed of keypoint alignment. Enumeration search needs to search all the templates in the library, so the time complexity depends on the total number  $n$  of templates in the library. GOV Sort Search limits the search space to a specific size  $num$ , so the time complexity is O(num).

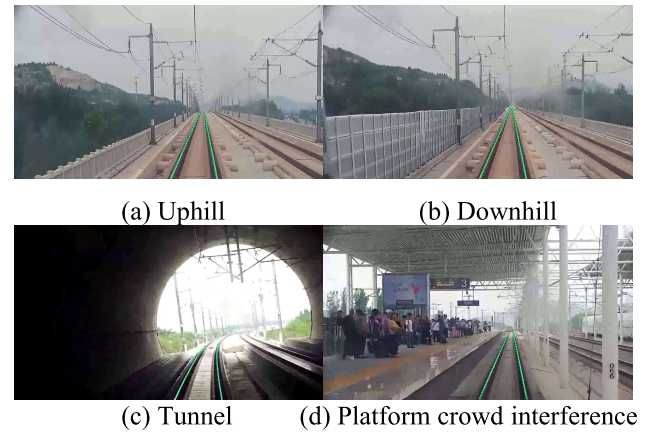


Fig. 16. Straight track detection results.

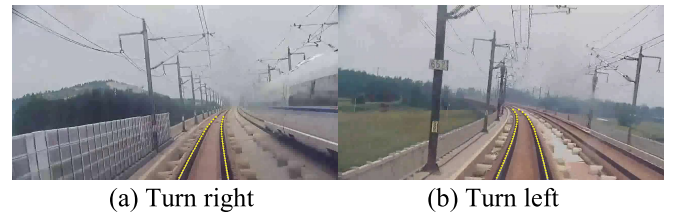


Fig. 17. Curved track detection results.

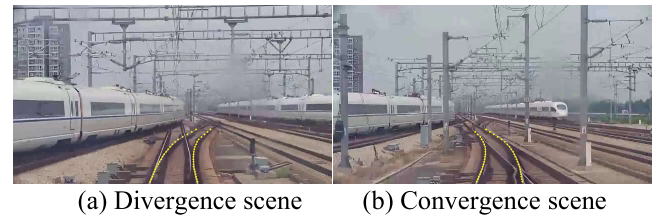


Fig. 18. Turnout detection results.

GOV sort search improves the detection speed greatly, and when  $num$  takes 18, FPS reaches 322.

The visualization of Track²Net is depicted in Fig. 16, 17, 18 and 19. It can be seen that Track²Net performs well regardless of uphill, downhill, tunnel, platform crowd interference, curved track and turnouts.

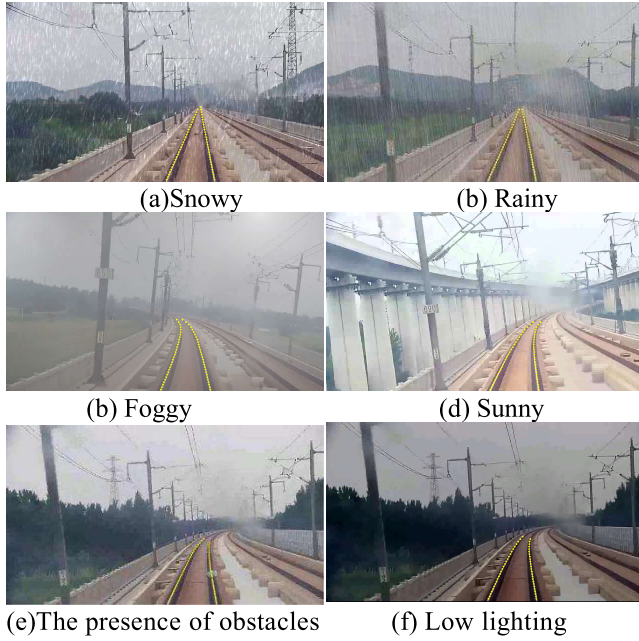


Fig. 19. Complex natural environment detection results.

TABLE V  
ABLATION EXPERIMENT

| Keypoint Alignment | CTPM | Posting treatment | $PIOU$ | $ACC_{point}$ (%) | $ACC_{line}$ (%) | $MR_{point}$ (%) | FPS |
|--------------------|------|-------------------|--------|-------------------|------------------|------------------|-----|
| ✓                  | ✓    | ✓                 | ✓      | 90.95             | 91.48            | 0.96             | 236 |
| ✓                  | ✓    | ✓                 |        | 89.05             | 90.12            | 1.04             | 237 |
| ✓                  | ✓    |                   |        | 89.05             | 89.32            | 1.04             | 237 |
|                    | ✓    |                   |        | 87.04             | 87.88            | 2.87             | 159 |
| ✓                  |      |                   |        | 82.58             | 80.13            | 1.94             | 322 |

### E. Ablation Experiment

In this paper, the influence of each component in the model on performance is analyzed in depth through ablation experiments, as shown in TABLE V. The experiment uses ResNet34 as the backbone of feature extraction, showing a processing speed of 90.95 % and 236 FPS. By gradually removing different modules in the model, it is observed that the removal of  $L_{PIOU}$  results in a decrease in  $ACC_{point}$  to 89.05 %. The removal of the post-treatment module has no effect on  $ACC_{point}$ , but  $ACC_{line}$  drops significantly. This shows that the module is very important to detect the track position at the turnout. The keypoint alignment and the CTPM, as the most important modules in this paper, both have the ability to detect the track line independently. When the CTPM is used exclusively for track detection, the speed is significantly reduced, resulting in an FPS of only 159. If only keypoint alignment is employed to detect the tracks, FPS reaches 322, but  $ACC_{point}$  is only 82.5. The experimental results indicate

TABLE VI  
USAGE RATIO OF KEYPOINT ALIGNMENT AND CTPM ON HRTD

| Method              | Usage rate (%) |
|---------------------|----------------|
| keypoints alignment | 53.69          |
| CTPM                | 47.31          |

TABLE VII  
KEYPOINT ALIGNMENT DETECTION EFFECT ON HRTD

| Datasets          | $ACC_{point}$ (%) | $ACC_{line}$ (%) | $MR_{point}$ (%) | FPS  |
|-------------------|-------------------|------------------|------------------|------|
| Straight Rail     | 96.29             | 98.79            | 0.45             | 1006 |
| Non-straight Rail | 70.91             | 44.67            | 3.26             | 322  |

TABLE VIII  
COMPARISON BETWEEN KNCC AND OTHER TEMPLATE MATCHING ALGORITHMS ON CWTD

| Method             | $ACC_{point}$ (%) | FPS |
|--------------------|-------------------|-----|
| Absolute distance  | 68.59             | 487 |
| Euclidean distance | 71.22             | 395 |
| KNCC               | 78.94             | 322 |

that a higher level of speed and accuracy is attainable only through the combination of keypoint alignment and CTPM.

Table VI shows the usage ratio of the keypoint alignment and the CTPM. Both account for about 50%, which proves the importance of the two modules. This also reflects that the two methods have different tendencies for different scenarios.

In order to further highlight the superiority of the keypoint match method in a simple scene, we also conduct further experiments under straight rail datasets, as shown in TABLE VII. For non-straight rails with complex scenes, performance is poor. However, the performance of this method is very superior in the straight rail scene. Here, only five straight rail templates are required.

Table VIII gives a comparison of template matching algorithms on CWTD. The template matching method based on absolute distance reaches 68.59% accuracy at 487 FPS, and the template matching method based on Euclidean distance reaches 71.22% accuracy at 395 FPS. In contrast, KNCC achieves 82.94% accuracy at 322 FPS. The template matching algorithm based on KNCC is far higher than other methods in accuracy while ensuring a high detection speed. This shows that KNCC has good robustness to different weather and illumination.

Table IX indicates the influence of using different transition construction templates on the Keypoint Alignment method. The accuracy of cubic parabola is 82.58%, which is higher than that of half-wave sine curve.

TABLE IX  
INFLUENCE OF DIFFERENT TRANSITION  
CURVES

| Curve                | $ACC_{point}$ (%) |
|----------------------|-------------------|
| Cubic parabola       | 82.58             |
| Half-wave sine curve | 82.07             |

#### IV. CONCLUSION

A real-time online monitoring algorithm Track2Net for track line tracking based on monocular images is proposed. The proposed model addresses tracks detection employing a combination of keypoint alignment and track anchor identification. Meanwhile, this paper constructs the track datasets of the train operation. The experiments in the datasets demonstrate that the lightweight version of Track<sup>2</sup>Net achieves a speed of 243 FPS while maintaining high accuracy. The ResNet34 version of Track<sup>2</sup>Net shows that the line accuracy rate reaches 91.43 %, and the FPS reaches 236. Track<sup>2</sup>Net consistently delivers high detection accuracy for various scenes such as uphill, downhill, tunnels and turnouts, which can meet the real-time tracking of the track line in the high-speed railway scene.

As the key equipment of the railway, the tracks have complex structures and different forms. Although the results of the above verification are promising, there are some limitations. The proposed method is only performed when the relative position between the camera and the ground is fixed (or the deviation is not large). In the future, we will explore the adaptability of the algorithm under different types of tracks and different speeds to achieve a wider range of application scenarios.

#### REFERENCES

- [1] F. Kaleli and Y. S. Akgul, "Vision-based railroad track extraction using dynamic programming," in *Proc. 12th Int. IEEE Conf. Intell. Transp. Syst.*, Oct. 2009, pp. 1–6, doi: [10.1109/ITSC.2009.5309526](#).
- [2] J. Wohlfeil, "Vision based rail track and switch recognition for self-localization of trains in a rail network," in *Proc. IEEE Intell. Vehicles Symp. (IVS)*, Jun. 2011, pp. 1025–1030, doi: [10.1109/IVS.2011.5940466](#).
- [3] J. C. Espino and B. Stanculescu, "Rail extraction technique using gradient information and a priori shape model," in *Proc. 15th Int. IEEE Conf. Intell. Transp. Syst.*, Oct. 2012, pp. 1132–1136, doi: [10.1109/ITSC.2012.6338870](#).
- [4] Z. Qi, Y. Tian, and Y. Shi, "Efficient railway tracks detection and turnouts recognition method using HOG features," *Neural Comput. Appl.*, vol. 23, no. 1, pp. 245–254, Jul. 2013, doi: [10.1007/s00521-012-0846-0](#).
- [5] M. H. Zwemer et al., "A vision-based approach for tramway rail extraction," *Proc. SPIE*, vol. 9407, pp. 227–239, Oct. 2015, doi: [10.1117/12.2075641](#).
- [6] M. A. Selver, E. Y. Zoral, B. Belenlioglu, and S. Dogan, "Predictive modeling for monocular vision based rail track extraction," in *Proc. 10th Int. Congr. Image Signal Process., Biomed. Eng. Informat. (CISP-BMEI)*, Shanghai, China, Oct. 2017, pp. 1–6, doi: [10.1109/CISP-BMEI.2017.8301928](#).
- [7] Y.-S. Zheng, Y.-W. Jin, and Y. Dong, "Rail detection based on LSD and the least square curve fitting," *Int. J. Autom. Comput.*, vol. 18, no. 1, pp. 85–95, Feb. 2021, doi: [10.1007/s11633-020-1241-4](#).
- [8] Z. Teng, F. Liu, and B. Zhang, "Visual railway detection by superpixel based intracellular decisions," *Multimedia Tools Appl.*, vol. 75, no. 5, pp. 2473–2486, May 2015, doi: [10.1007/s11042-015-2654-x](#).
- [9] D. Yu and G. Bi, "Railway track detection algorithm based on hu invariant moment feature," *J. China Railway Soc.*, vol. 10, no. 10, pp. 64–70, 2018.
- [10] B. T. Nassu and M. Ukai, "A vision-based approach for rail extraction and its application in a camera pan-tilt control system," *IEEE Trans. Intell. Transp. Syst.*, vol. 13, no. 4, pp. 1763–1771, Dec. 2012, doi: [10.1109/TITS.2012.2204052](#).
- [11] Z. Wang, X. Wu, G. Yu, and M. Li, "Efficient rail area detection using convolutional neural network," *IEEE Access*, vol. 6, pp. 77656–77664, 2018, doi: [10.1109/ACCESS.2018.2883704](#).
- [12] Y. Wang, L. Wang, Y. H. Hu, and J. Qiu, "RailNet: A segmentation network for railroad detection," *IEEE Access*, vol. 7, pp. 143772–143779, 2019, doi: [10.1109/ACCESS.2019.2945633](#).
- [13] S. Belyaev, I. Popov, V. Shubnikov, P. Popov, E. Boltenkova, and D. Savchuk, "Railroad semantic segmentation on high-resolution images," in *Proc. IEEE 23rd Int. Conf. Intell. Transp. Syst. (ITSC)*, Sep. 2020, pp. 1–6, doi: [10.1109/ITSC45](#).
- [14] M. Ghorbanalivakili, J. Kang, G. Sohn, D. Beach, and V. Marin, "TPE-Net: Track point extraction and association network for rail path proposal generation," in *Proc. IEEE 19th Int. Conf. Autom. Sci. Eng. (CASE)*, Aug. 2023, pp. 1–7, doi: [10.1109/CASE56687.2023.10260541](#).
- [15] X. Pan, J. Shi, P. Luo, X. Wang, and X. Tang, "Spatial as deep: Spatial CNN for traffic scene understanding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, Apr. 2018, pp. 7276–7283, doi: [10.1609/aaai.v32i1.12301](#).
- [16] Z. Tu et al., "RESA: Recurrent feature-shift aggregator for lane detection," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 4, pp. 3547–3554.
- [17] H. Xu, S. Wang, X. Cai, W. Zhang, X. Liang, and Z. Li, "CurveLane-NAS: Unifying lane-sensitive architecture search and adaptive point blending," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 689–704, doi: [10.1007/978-3-030-58555-6\\_41](#).
- [18] L. Tabelini, R. Berriel, T. M. Paixão, C. Badue, A. F. De Souza, and T. Oliveira-Santos, "PolyLaneNet: Lane estimation via deep polynomial regression," in *Proc. 25th Int. Conf. Pattern Recognit. (ICPR)*, Milan, Italy, Jan. 2021, pp. 6150–6156, doi: [10.1109/ICPR48806.2021.9412265](#).
- [19] R. Liu, Z. Yuan, T. Liu, and Z. Xiong, "End-to-end lane shape prediction with transformers," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 3693–3701, doi: [10.1109/WACV48630.2021.00374](#).
- [20] S. Yang, G. Yu, Z. Wang, B. Zhou, P. Chen, and Q. Zhang, "A topology guided method for rail-track detection," *IEEE Trans. Veh. Technol.*, vol. 71, no. 2, pp. 1426–1438, Feb. 2022, doi: [10.1109/TVT.2021.3133327](#).
- [21] X. Li, J. Li, X. Hu, and J. Yang, "Line-CNN: End-to-end traffic line detection with line proposal unit," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 1, pp. 248–258, Jan. 2020, doi: [10.1109/TITS.2019.2890870](#).
- [22] L. Tabelini, R. Berriel, T. M. Paixão, C. Badue, A. F. D. Souza, and T. Oliveira-Santos, "Keep your eyes on the lane: Real-time attention-guided lane detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 294–302, doi: [10.1109/CVPR46437.2021.00036](#).
- [23] Z. Qin, H. Wang, and X. Li, "Ultra fast structure-aware deep lane detection," in *Proc. 16th Eur. Conf. Comput. Vis.-ECCV*, 2020, pp. 276–291, doi: [10.1007/978-3-030-58586-0\\_17](#).
- [24] L. Liu, X. Chen, S. Zhu, and P. Tan, "CondLaneNet: A top-to-down lane detection framework based on conditional convolution," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 3753–3762.
- [25] T. Zheng et al., "CLRNet: Cross layer refinement network for lane detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 888–897, doi: [10.1109/CVPR52688](#).
- [26] J. Wang et al., "A keypoint-based global association network for lane detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 1382–1391, doi: [10.1109/CVPR52688.2022.00145](#).
- [27] *Code for Design of High Speed Railway*, The Third Railway Survey and Design Institute Group Corporation, Beijing, China, 2014.
- [28] S. Hong and D. Park, "Runtime virtual lane prediction based on inverse perspective transformation and machine learning for lane departure warning in low-power embedded systems," in *Proc. IEEE Int. Conf. Imag. Syst. Techn. (IST)*, Kaohsiung, Taiwan, Jun. 2022, pp. 1–6, doi: [10.1109/IST55454.2022.9827740](#).
- [29] S. Zhenyu, W. Lihui, and L. Liqiang, "Three-dimensional reconstruction algorithm based on inverse perspective transformation," in *Proc. Int. Conf. Big Data, Artif. Intell. Internet Things Eng. (ICBAIE)*, Fuzhou, China, Jun. 2020, pp. 221–225, doi: [10.1109/ICBAIE49996.2020.00053](#).

- [30] X. Gao, K. Zeng, M. Hu, D. Zhao, and M. Wu, "Distance measurement method based on two-dimensional image recognition and inverse perspective transformation," in *Proc. IEEE Asia-Pacific Conf. Image Process., Electron. Comput. (IPEC)*, Dalian, China, Apr. 2021, pp. 604–607, doi: [10.1109/IPEC51340.2021.9421144](https://doi.org/10.1109/IPEC51340.2021.9421144).
- [31] X. Liu, Z. M. Cheng, and F. Y. Yi, "A road detection technology based on reverse perspective transformation," in *Proc. 8th Int. Conf. Intell. Hum.-Mach. Syst. Cybern. (IHMSC)*, Aug. 2016, pp. 126–129, doi: [10.1109/IHMSC.2016.24](https://doi.org/10.1109/IHMSC.2016.24).
- [32] C. Xiu and X. Pan, "Tracking algorithm based on the improved template matching," in *Proc. 29th Chin. Control Decis. Conf. (CCDC)*, Chongqing, China, May 2017, pp. 483–486, doi: [10.1109/CCDC.2017.7978142](https://doi.org/10.1109/CCDC.2017.7978142).
- [33] J. Y. Zhang, Z.-Q. Liu, and Y. Liu, "Research on the fast template matching method of micro parts," in *Proc. 7th Int. Conf. Inf. Sci. Control Eng. (ICISCE)*, Dec. 2020, pp. 2151–2156, doi: [10.1109/ICISCE50968.2020.00422](https://doi.org/10.1109/ICISCE50968.2020.00422).
- [34] X. Qi and L. Miao, "A template matching method for multi-scale and rotated images using ring projection vector conversion," in *Proc. IEEE 3rd Int. Conf. Image, Vis. Comput. (ICIVC)*, Chongqing, China, Jun. 2018, pp. 45–49, doi: [10.1109/ICIVC.2018.8492726](https://doi.org/10.1109/ICIVC.2018.8492726).
- [35] D. Song, L. Liu, X. Zhang, K. Qin, and L. Wang, "A novel hog-based template matching method for SAR and optical image," in *Proc. IGARSS-IEEE Int. Geosci. Remote Sens. Symp.*, Kuala Lumpur, Malaysia, Jul. 2022, pp. 951–954, doi: [10.1109/IGARSS46834.2022.9884706](https://doi.org/10.1109/IGARSS46834.2022.9884706).
- [36] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778, doi: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- [37] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020, doi: [10.1109/TPAMI.2018.2858826](https://doi.org/10.1109/TPAMI.2018.2858826).
- [38] J. C. Espino, B. Stanculescu, and P. Forin, "Rail and turnout detection using gradient information and template matching," in *Proc. IEEE Int. Conf. Intell. Rail Transp.*, Oct. 2013, pp. 233–238, doi: [10.1109/ICIRT.2013.6696299](https://doi.org/10.1109/ICIRT.2013.6696299).
- [39] C. Tastimur, O. Yaman, M. Karakose, and E. Akin, "A real time interface for vision inspection of rail components and surface in railways," in *Proc. Int. Artif. Intell. Data Process. Symp. (IDAP)*, Malatya, Turkey, Sep. 2017, pp. 1–6, doi: [10.1109/IDAP.2017.8090267](https://doi.org/10.1109/IDAP.2017.8090267).
- [40] Y. Gao, Y. Yang, Y. Ma, and W. Xu, "Research on intelligent diagnosis of railway turnout based on FastDTW under big data monitoring," in *Proc. Int. Conf. Inf. Control, Electr. Eng. Rail Transit (ICEERT)*, Lanzhou, China, Oct. 2021, pp. 283–286, doi: [10.1109/ICEERT53919.2021.00059](https://doi.org/10.1109/ICEERT53919.2021.00059).
- [41] Z. Cao, Y. Gao, J. Bai, Y. Qin, Y. Zheng, and L. Jia, "Efficient dual-stream fusion network for real-time railway scene understanding," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 9442–9452, Aug. 2024, doi: [10.1109/TITS.2024.3377187](https://doi.org/10.1109/TITS.2024.3377187).
- [42] T. Sun, B. Guo, T. Ruan, X. Zhou, and D. Bai, "RailEINet: A novel scene segmentation network for automatic train operation based on feature alignment," *Eng. Appl. Artif. Intell.*, vol. 138, Dec. 2024, Art. no. 109295, doi: [10.1016/j.engappai.2024.109295](https://doi.org/10.1016/j.engappai.2024.109295).
- [43] L. Lian et al., "Densely multiscale fusion network for lightweight and accurate semantic segmentation of railway scenes," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–11, 2024, doi: [10.1109/TIM.2024.3413143](https://doi.org/10.1109/TIM.2024.3413143).



**Xiukun Wei** (Member, IEEE) received the Ph.D. degree from Johannes Kepler University, Linz, Austria, in 2002. From 2002 to 2006, he was a Research Assistant with the Institute of Design and Control of Mechatronic Systems, Johannes Kepler University. From 2006 to 2009, he was a Post-Doctoral Researcher with Delft Center for Systems and Control, Delft University of Technology, Delft, The Netherlands. He is currently a Professor with the Frontiers Science Center for Smart High-Speed Railway System and the School of Traffic and Transportation, Beijing Jiaotong University, Beijing, China. His research interests include machine vision, fault diagnosis and its applications, intelligent transportation systems, and condition monitoring and its applications in a variety of fields, such as rail traffic control, safety, and transportation.



**Yang Ji** received the bachelor's degree from Beijing Jiaotong University, where he is currently pursuing the degree in control science and engineering. His research interests include machine vision, fault diagnosis, system security, and deep learning methods and their applications.



**Chengyu Ge** received the bachelor's degree from Beijing Jiaotong University, China, where he is currently pursuing the degree in control science and engineering. His research interests include image detection, point cloud detection, and deep learning methods and their applications.



**Qingfeng Tang** received the bachelor's degree from Beijing Jiaotong University, Beijing, China, where he is currently pursuing the Ph.D. degree with the School of Traffic and Transportation. His research interests include fault diagnosis, machine vision, and deep learning method and their applications.



**Haipeng Guo** is currently pursuing the Ph.D. degree in safety science and engineering with the School of Traffic and Transportation, Beijing Jiaotong University.

His research interests include image processing, deep learning, rail transit safety detection, and their applications.



**Kunlin Cai** received the bachelor's degree from Beijing Jiaotong University, China, in 2022, where he is currently pursuing the master's degree with the School of Traffic and Transportation. His research interests include computer vision and deep learning.